

COMPSYS302: Python

6. Control Flow

Ho Seok AHN

hs.ahn@auckland.ac.nz

6.1 Control Flow Statements

□ Control flow

- A program's **control flow** is the order in which the program's code executes.
- The **control flow** of a Python program is regulated by conditional statements, loops, and function calls.
- The followings are very important control flows in Python.
 - if / elif / else
 - while
 - for
 - break
 - continue
 - switch: Python does not have this statement

6.2 The if Statement

□ The if Statement

- Often, you need to execute some statements only if some condition holds, or choose statements to execute depending on several mutually exclusive conditions.
- The Python compound statement **if** lets you conditionally execute blocks of statements. The syntax is:

if expression:
 statement(s)

if expression: statement

- Ex) 6_01_If.py

```
a = input("input any number: ")
```

```
if int(a) > 0:
```

```
    result = "a is positive"
```

```
    print (result)
```

```
if int(a) > 0: print ("a is positive")
```

(result)

input any number: 3

a is positive

a is positive

6.2 The if Statement

□ The elif / else Statement

- You can use multiple conditions using **elif** and **else**.
- The syntax is:

```
if expression:  
    statement(s)  
elif expression:  
    statement(s)  
else:  
    statement(s)
```

- Ex) 6_02_Elif.py

```
a = input("input any number: ")  
if int(a) > 0:  
    print ("a is positive")  
elif int(a) < 0:  
    print ("a is negative")  
else:  
    print ("a is zero")
```

(result)

input any number: -1

a is negative

6.2 The if Statement

□ The elif / else Statement

- You can use multiple expressions for one condition.

- Ex) 6_03_MultipleExpressions.py

```
score = input("input your score: ")
if 90 <= int(score):
    grade = "A"
elif 80 <= int(score) < 90:
    grade = "B"
elif 70 <= int(score) < 80:
    grade = "C"
elif 60 <= int(score) < 70:
    grade = "D"
else:
    grade = "F"
print("Your grade is " + grade)
```

(result)

input your score: 99

Your grade is A

6.3 The while Statement

□ The while Statement

- The **while** statement in Python supports repeated execution of a statement or a block of statements that is controlled by a conditional expression.
- The syntax is:

while expression:
statement(s)

- Ex) 6_04_While.py

```
a = input("insert any integer: ")
```

```
value = 0
```

```
while value < int(a):
```

```
    print (value)
```

```
    value += 1
```

```
print ("out of the while loop")
```

(result)

insert any integer: 5

0

1

2

3

4

out of the while loop

6.4 The for Statement

□ The for Statement

- The **for** statement in Python supports repeated execution of a statement or block of statements that is controlled by an iterable expression.
- The syntax is:

for target in iterable_object:
 statement(s)

- Ex) 6_05_For.py

```
# list
l = ['apple', 100, 12.34]
for i in l:
    print(i, type(i))
print("\ t")
# dictionary
d = {'apple':1, 'orange':2, 'kiwi':3}
for key, value in d.items():
    print(key + " is $" + str(value) + " per kg")
```

(result)
apple <type 'str'>
100 <type 'int'>
12.34 <type 'float'>

orange is \$2 per kg
kiwi is \$3 per kg
apple is \$1 per kg

6.4 The for Statement

□ Iterators

- An **iterator** is an object that enables a programmer to traverse a container.
- Python defines several **iterator** objects to support iteration over general and specific sequence types, dictionaries, and other more specialized forms.

- Ex) 6_06_Iterator.py

```
mytuple = ("apple", "banana", "cherry")  
myit = iter(mytuple)
```

```
print(next(myit))  
print(next(myit))  
print(next(myit))
```

```
(result)  
apple  
banana  
cherry
```


6.5 The break Statement

□ The break Statement

- The **break** statement is allowed only inside a loop body.
- When **break** executes, the loop terminates.

- Ex) 6_07_Break.py

```
l = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
for i in l:
```

```
    if i > 5:
```

```
        print ("break")
```

```
        break
```

```
    print (i)
```

(result)

1

2

3

4

5

break

6.6 The continue Statement

□ The continue Statement

- The **continue** statement is allowed only inside a loop body.
- When **continue** executes, the current iteration of the loop body terminates, and execution continues with the next iteration of the loop.

- Ex) 6_08_Continue.py

```
l = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
for i in l:
```

```
    if i % 2 == 0:
```

```
        print ("continue")
```

```
        continue
```

```
    print (i)
```

(result)

1

continue

3

continue

5

continue

7

continue

9